Lab: Generative AI for Models Development

Estimated time: 30 minutes

In this lab, we will use generative AI to create Python scripts to develop and evaluate different predictive models for a given data set.

Learning objectives

In this lab, you will learn how to use generative AI to create Python codes that can:

- Use linear regression in one variable to fit the parameters to a model
- Use linear regression in multiple variables to fit the parameters to a model
- Use polynomial regression in a single variable to fit the parameters to a model
- Create a pipeline for performing linear regression using multiple features in polynomial scaling
- Use the grid search with cross-validation and ridge regression to create a model with optimum hyperparameters

About generative AI classroom lab

▶ Click here

Notes:

1. The prompts used in this lab are for your reference only. You can create your own prompts and generate responses using generative AI.
2. Since AI-generated outputs are dynamic, you may receive different responses even though you've used the same prompt from this lab.

Code execution environment

To test the prompt-generated code, keep the Jupyter Notebook (in the link below) open in a separate tab in your web browser. The notebook has some setup instructions that you should complete now.

Jupyter-Lite Test Environment

The data set for this lab is available in the following URL.

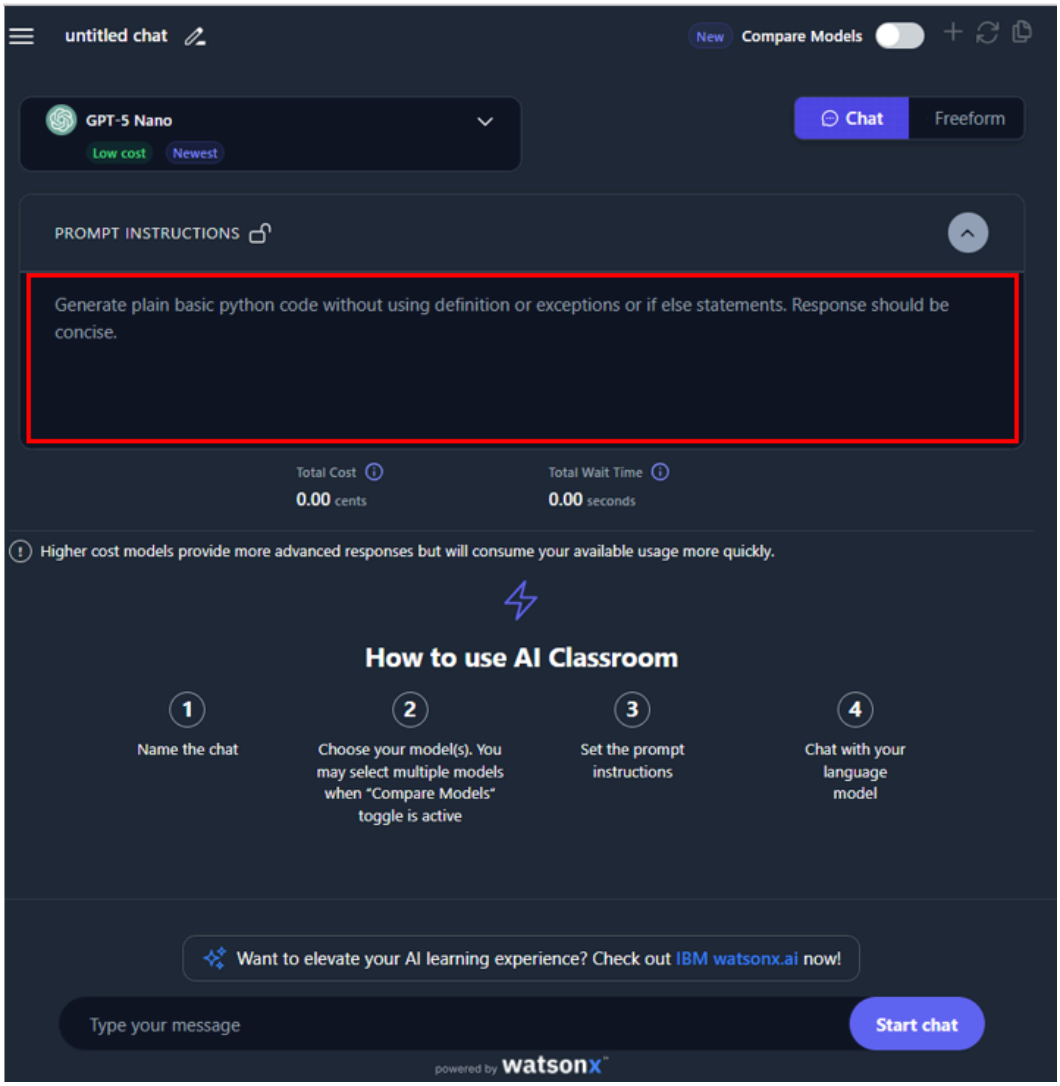
```
URL = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DA0101EN-Coursera/laptop_pricing_da
```

The dataset is a filtered and modified version of the Laptop Price Prediction using specifications dataset, available under the Database Contents License (DbCL) v1.0 on the Kaggle website.

Complete the setup in the Jupyter Notebook and then proceed further.

Building the prompt: Importing data set

Now you will start building your prompt. You can set the prompt like below to get a defined output:



The first step is to ask the Gen AI model to generate a code to import the provided dataset to a Pandas' data frame. You must specify if you imported the data. Then, you should see the dataset headers in the first row of the CSV file.

You can structure the prompt to create the code as follows:

```
Write a Python code that can perform the following tasks.
Read the CSV file, located on a given file path, into a pandas data frame, assuming that the first row of the file can be used as the h
```

Running this prompt using the **GPT-5 Nano** engine in the GenAI classroom should result in a response that contains the code, as shown below.

```
Reads a CSV from the provided file path into a pandas DataFrame, using the first row as column headers.
import pandas as pd
file_path = "/path/to/your/file.csv"
df = pd.read_csv(file_path, header=0)
```

You may copy the code from the response and try it on the Jupyter Notebook with appropriate modifications in the file path.

Linear regression in one variable

You can now ask the generative AI model to generate a script to create a linear regression model using a single attribute as the training feature to predict a target attribute.

You can write the prompt as follows:

```
Write a Python code that performs the following tasks.
1. Develops and trains a linear regression model that uses one attribute of a data frame as the source variable and another as a target
2. Calculate and display the MSE and R^2 values for the trained model.
```

It will generate a response as shown below:

```
Reads a CSV into a DataFrame, selects one feature as X and one as y, trains a simple linear regression, and computes MSE and R^2 on the
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
# Configuration: adjust these for your data
file_path = "/path/to/your/file.csv"  # CSV with header row
feature_col = "feature"  # source variable (X)
target_col = "target"    # target variable (y)
# Load data
df = pd.read_csv(file_path, header=0)
# Prepare features and target
X = df[[feature_col]]
y = df[target_col]
# Train model
model = LinearRegression()
model.fit(X, y)
# Predict and evaluate
y_pred = model.predict(X)
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
# Output results
print("MSE:", mse)
print("R^2:", r2)
```

You can use this code to develop a linear regression model with the target variable as `Price` and the source variable as `CPU_frequency`. Try this out in the Test environment.

Linear regression in multiple variables

You can modify the prompt to ask for a code that uses multiple attributes as input for training a model to predict the value of a target variable.

For that, you can use the prompt as shown below:

```
Write a Python code that performs the following tasks.
1. Develops and trains a linear regression model that uses some attributes of a data frame as the source variables and one of the attri
2. Calculate and display the MSE and R^2 values for the trained model.
```

You should see a similar response to the following:

```
Trains a linear regression model using all non-target columns as features, then computes MSE and R^2 on the training data.
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
# Assumes an existing DataFrame named 'df' with a target column named 'target'
# All other columns are treated as features
target_col = "target"
feature_cols = df.columns.drop(target_col)
X = df[feature_cols]
y = df[target_col]
# Train linear regression model
model = LinearRegression()
model.fit(X, y)
# Predict on training data and evaluate
y_pred = model.predict(X)
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
print("MSE:", mse)
print("R^2:", r2)
```

You can use the generated code and build a linear regression model for the data set that uses `CPU_frequency`, `RAM_GB`, `Storage_GB_SSD`, `CPU_core`, `OS`, `GPU` and `Category` and source variables and `Price` as the target variable. You can check the code generated in the webpage browser.

You may compare the performance of the two models by comparing their MSE and `R^2` values.

Polynomial regression

You can explore creating a model that uses higher-order features from original attributes. Higher orders of the same feature allow better generalization on the target variable. It is called polynomial regression, and you can use Generative AI to create a code for this.

Assume you are given a single attribute as the source variable and one as a target variable. You must create a model using polynomial regression for a given order. You can also make the model for different order values and compare their performance based on MSE and R^2 scores.

You can write a similar prompt to the following code:

```
Write a Python code that performs the following tasks.
1. Develops and trains multiple polynomial regression models, with orders 2, 3, and 5, that use one attribute of a data frame as the so
2. Calculate and display the MSE and R^2 values for the trained models.
3. Compare the performance of the models.
```

The expected output should look as shown below:

```
The code trains polynomial regression models of degrees 2, 3, and 5 on a single feature to predict a target. It computes MSE and R^2 fo
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
# Configuration: adjust file path and column names as needed
file_path = "/path/to/your/file.csv"  # CSV with header row
feature_col = "feature"  # source variable (X)
target_col = "target"    # target variable (y)
# Load data
df = pd.read_csv(file_path, header=0)
# Prepare single-feature input
X = df[[feature_col]]
y = df[target_col]
# Degrees for polynomial regression
degrees = [2, 3, 5]
results = []
for degree in degrees:
```

```
    # Generate polynomial features for the single input feature
    poly = PolynomialFeatures(degree=degree, include_bias=False)
    X_poly = poly.fit_transform(X)
    # Train linear regression on the expanded features
    model = LinearRegression()
    model.fit(X_poly, y)
    # Evaluate on training data
    y_pred = model.predict(X_poly)
    mse = mean_squared_error(y, y_pred)
    r2 = r2_score(y, y_pred)
    results.append({"degree": degree, "mse": mse, "r2": r2})
# Best model by R^2
best = max(results, key=lambda r: r["r2"])
# Display results
print("Results by degree:")
for r in results:
    print(f"Degree {r['degree']}: MSE={r['mse']:.6f}, R2={r['r2']:.6f}")
print("Best model: degree", best["degree"], "R2=", best["r2"])
```

You can use the relevant part of the code in your script.

You can see that the model can generate sophisticated code using functions to create and train models with different orders and evaluate their performance for each of them.

Try to run the generated code on the testing interface with the source variable as `CPU frequency` and the target variable as `Price`.

Creating a Pipeline

Pipelines are processes containing a sequence of steps that lead to creating a trained model.

You will now use the Generative AI model to create a pipeline for performing feature scaling, creating polynomial features for multiple attributes, and performing linear regression using these variables.

You can build a similar prompt to the following code:

```
Write a Python code that performs the following tasks.
1. Create a pipeline that performs parameter scaling, Polynomial Feature generation, and Linear regression. Use the set of multiple fea
2. Calculate and display the MSE and R^2 values for the trained model.
```

The expected response is as shown below:

```
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
# Configuration: adjust as needed
file_path = "/path/to/your/file.csv"  # CSV with header row
target_col = "target"                 # name of the target column
# Load data
df = pd.read_csv(file_path, header=0)
# Validate columns
if target_col not in df.columns:
    raise ValueError(f"Target column '{target_col}' not found in data. Columns: {list(df.columns)}")
feature_cols = df.columns.drop(target_col)
if len(feature_cols) == 0:
    raise ValueError("No feature columns found besides the target column.")
X = df[feature_cols]
y = df[target_col]
# Create a pipeline: scale features, generate polynomial features, then fit linear regression
pipeline = Pipeline([
    ("scaler", StandardScaler()),
```

```
    ("poly", PolynomialFeatures(degree=2, include_bias=False)),
    ("model", LinearRegression())
])
# Train model
pipeline.fit(X, y)
# Evaluate on training data
y_pred = pipeline.predict(X)
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
print("MSE:", mse)
print("R^2:", r2)
```

You can use the relevant part in your code.

Make appropriate changes to the attributes you will use in the code. Consider the same set of attributes as used for multiple feature linear regression, which is `CPU_frequency`, `RAM_GB`, `Storage_GB_SSD`, `CPU_core`, `OS`, `GPU` and `Category` as source variables, and `Price` as the target variable.

Grid search and Ridge regression

An improved way to train your model is to use ridge regression instead of linear regression. You can use the polynomial features of multiple attributes. One of the key factors of ridge regression is using the parameter `alpha` as a hyperparameter for training. Using grid search, one can determine the optimum value of the hyperparameter for the given set of features. Grid search also uses cross-validation training to train and prepare the optimum model.

You can use generative AI to create the Python code to perform a grid search for the optimum ridge regression model, which uses polynomial features generated from multiple parameters.

You may use the following prompt to create this:

```
Write a Python code that performs the following tasks.
1. Use polynomial features for some of the attributes of a data frame.
2. Perform Grid search on a ridge regression model for a set of values of hyperparameter alpha and polynomial features as input.
3. Use cross-validation in the Grid search.
4. Evaluate the resulting model's MSE and R^2 values.
```

The resulting code will look as shown below:

```
import pandas as pd
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score
file_path = "/path/to/your/file.csv"
target_col = "target"
df = pd.read_csv(file_path, header=0)
feature_cols = df.columns.drop(target_col).tolist()
poly_features_cols = feature_cols[:2]
X = df[feature_cols]
y = df[target_col]
preprocessor = ColumnTransformer(
    transformers=[
        ("poly", PolynomialFeatures(degree=2, include_bias=False), poly_features_cols),
        ("passthrough", "passthrough", [c for c in feature_cols if c not in poly_features_cols])
    ]
)
pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("model", Ridge())
])
param_grid = {
    "preprocessor__poly__degree": [2, 3],
```

```
    "model__alpha": [0.1, 1.0, 10.0]
}
grid = GridSearchCV(pipe, param_grid, cv=5, scoring="neg_mean_squared_error", n_jobs=-1)
grid.fit(X, y)
best = grid.best_estimator_
y_pred = best.predict(X)
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
print("MSE:", mse)
print("R^2:", r2)
print("Best params:", grid.best_params_)
```

You can test this code for the data set on the testing environment.

You make use of the following parametric values for this purpose.

Source Variables: `CPU_frequency`, `RAM_GB`, `Storage_GB_SSD`, `CPU_core`, `OS`, `GPU` and `Category`
Target Variable: `Price`
Set of values for alpha: `0.0001,0.001,0.01, 0.1, 1, 10`
Cross Validation: 4-fold
Polynomial Feature order: 2

Conclusion

Congratulations! You have completed the lab on Data preparation.

With this, you have learned how to use generative AI to create Python codes that can:

- Implement linear regression in single variable
- Implement linear regression in multiple variables
- Implement polynomial regression for different orders of a single variable
- Create a pipeline that implements polynomial scaling for multiple variables and performs linear regression on them
- Apply a grid search to create an optimum ridge regression model for multiple features

Author(s)

Abhishek Gagneja